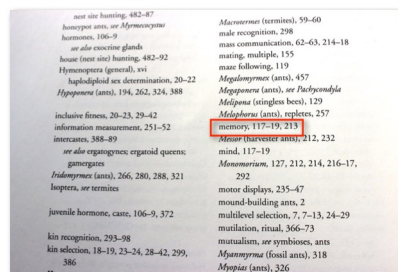


Pre-processing T

- If the Text will be searched many times, then there is very little cost of pre-processing
- An algorithm that pre-processes T is an *offline* algorithm, while one that does NOT pre-process it is an *online* algorithm.

149

Indexing



Key terms ordered alphabetically, with associated page #s



Grocery store items grouped into aisles

150

Indexing DNA

Index of T

T: C G T G C G T G C T T

151

Indexing DNA

Index of T

C G T G C : 0

T: C G T G C G T G C T T

152

Indexing DNA

Index of T
CGTGC: 0
GTGCG: 1

T: CGTGCGTGCTT

153

Indexing DNA

Index of T
CGTGC: 0
GTGCG: 1
TGC GT: 2

T: CGTGCGTGCTT

154

Indexing DNA

<i>Index of T</i>	
CGTGC:	0
→ GCGTG:	3
GTGCC:	1
TGCCT:	2

T: CGT GCGTG CTT

155

Indexing DNA

<i>Index of T</i>	
CGTGC:	0, 4
GCGTG:	3
GTGCC:	1
TGCCT:	2

T: CGT GCGTG CTT

156

Indexing DNA

Index of T

CGTGC: 0,4

GCGTG: 3

GTGCC: 1

GTGCT: 5

TGCCT: 2

T: CGTGCGTGCTT

157

Indexing DNA

Index of T

CGTGC: 0,4

GCGTG: 3

GTGCC: 1

GTGCT: 5

TGCCT: 2

TGCTT: 6

T: CGTGCGTGCTT

158

Indexing DNA

k-mer: substring
of length *k*

Index of T

CGTGC:	0, 4
GCGTG:	3
GTGCC:	1
GTGCT:	5
TGCCT:	2
TGCTT:	6

5-mer index

T: CGTGC GTGCTT

159

Querying the index

Index of T

CGTGC:	0, 4
GCGTG:	3
GTGCC:	1
GTGCT:	5
TGCCT:	2
TGCTT:	6

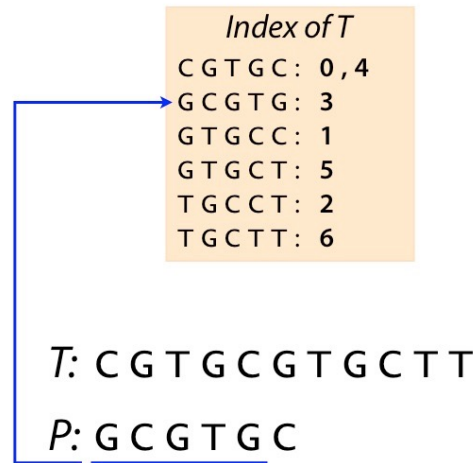
T: CGTGC GTGCTT

P: GCGTGC



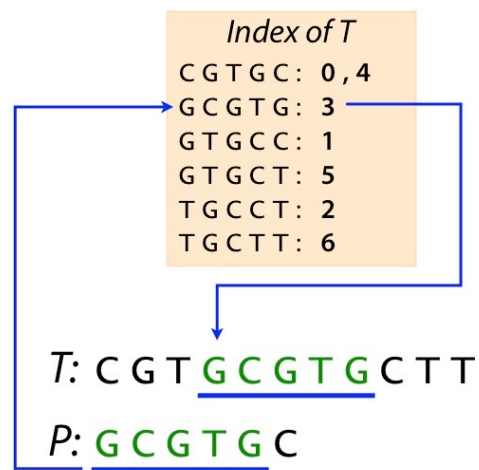
160

Querying the index



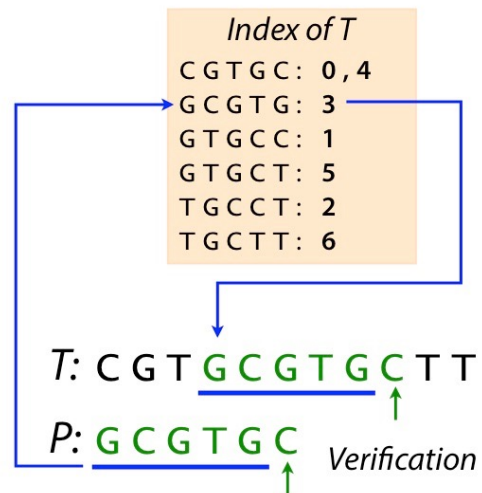
161

Querying the index



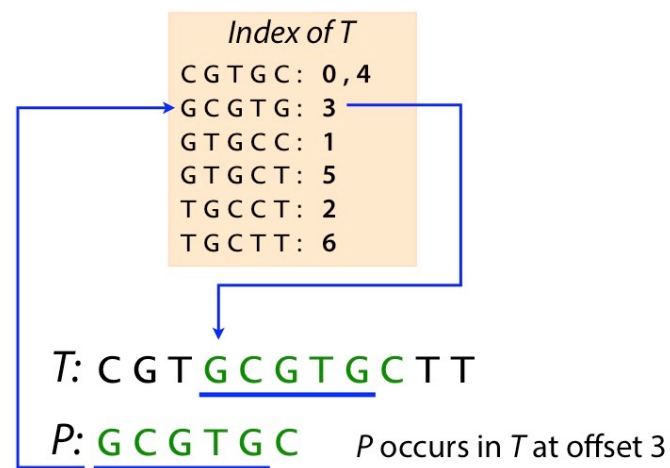
162

Querying the index



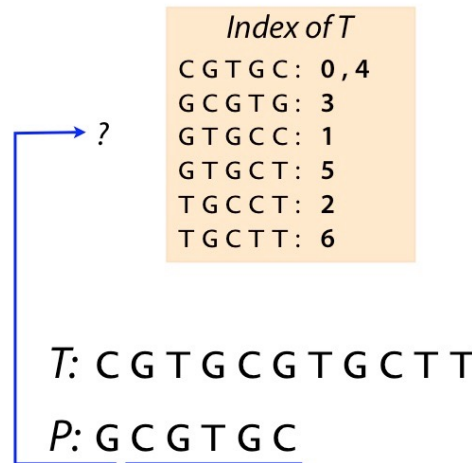
163

Querying the index



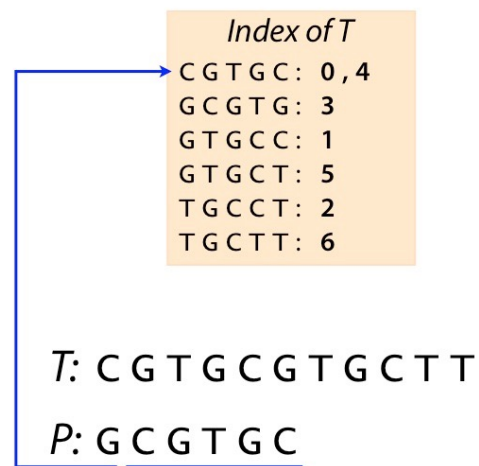
164

Querying the index



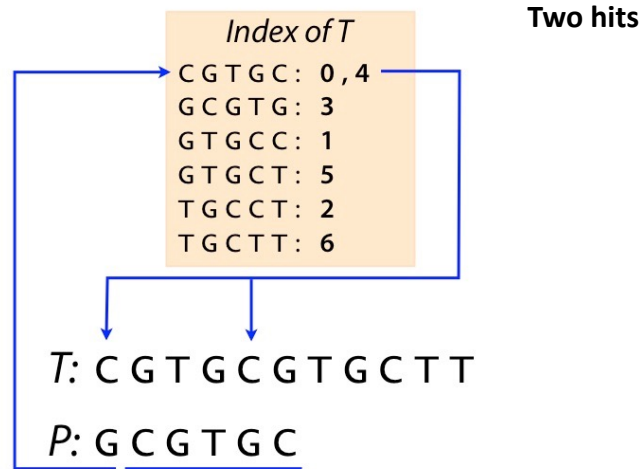
165

Querying the index



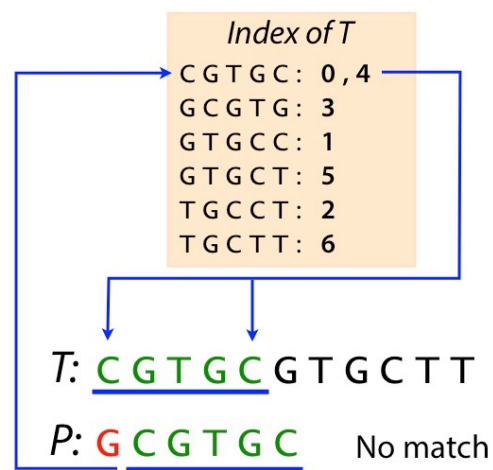
166

Querying the index



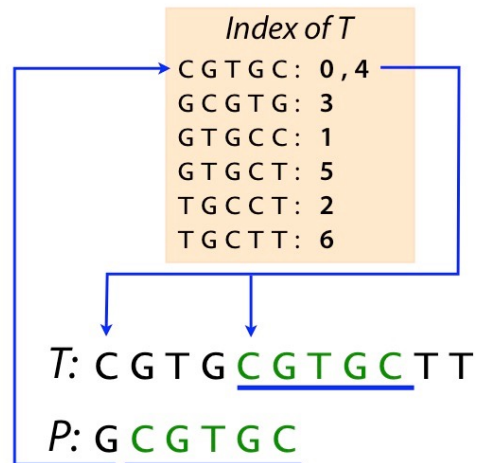
167

Querying the index



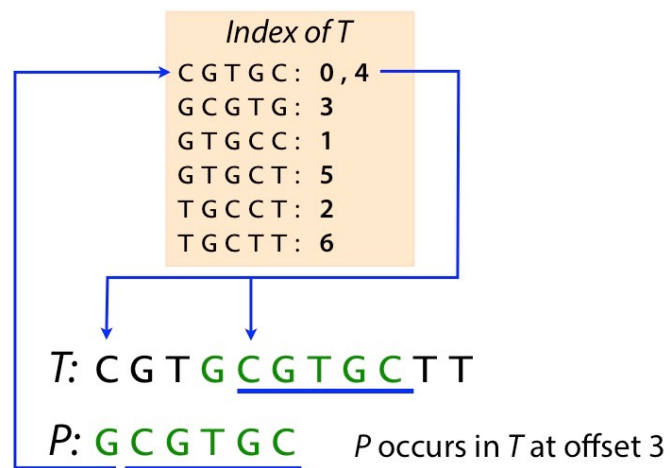
168

Querying the index



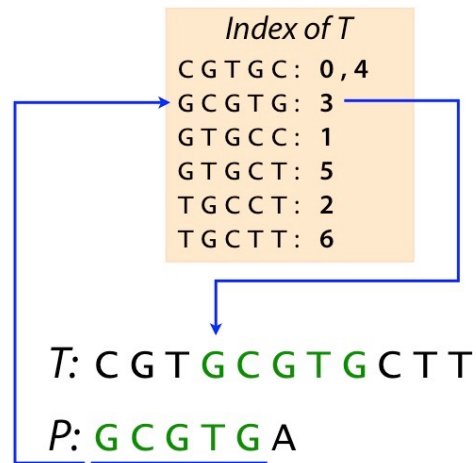
169

Querying the index



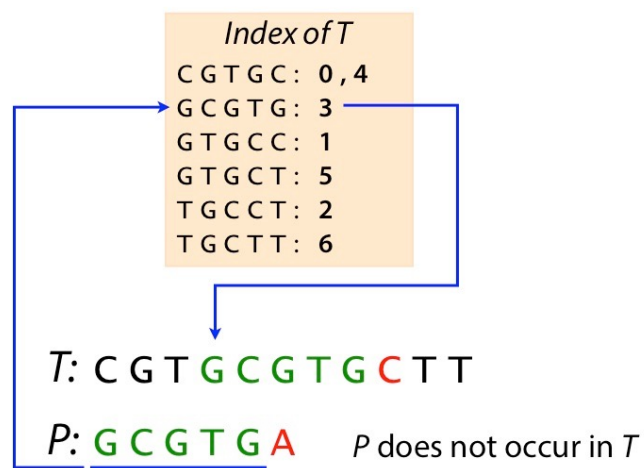
170

Querying the index



171

Querying the index



172

Querying the index

Index of <i>T</i>	
CGTGC:	0, 4
GCGTG:	3
GTGCC:	1
GTGCT:	5
TGCCT:	2
TGCTT:	6

T: CGTGCGTGCTT

P: GCGTAC



173

Querying the index

Index of <i>T</i>	
CGTGC:	0, 4
GCGTG:	3
GTGCC:	1
GTGCT:	5
TGCCT:	2
TGCTT:	6

T: CGTGCGTGCTT

P: GCGTAC *P* does not occur in *T*



174

Data structures

Index of T

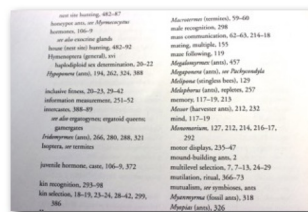
CGTGC: 0, 4
GCGTG: 3
GTGCC: 1
GTGCT: 5
TGCCT: 2
TGCTT: 6

Multimap

T : CGTGCGTGCTT

175

Data structures



Multimap

T : CGTGCGTGCTT

176

Data structures

G T G	0
T G C	1
G C G	2
C G T	3
G T G	4
T G T	5
G T G	6
T G G	7
G G G	8
G G G	9
G G G	10

T: G T G C G T G T G G G G G

177

Data structures

Alphabetical
by k-mer

C G T	3
G C G	2
G G G	8
G G G	9
G G G	10
G T G	0
G T G	4
G T G	6
T G C	1
T G G	7
T G T	5

T: G T G C G T G T G G G G G

178

Data structures

nest site hunting, 482–87	<i>Macrotermes</i> (termites), 59–60
honeypot ants, <i>see Myrmecocystus</i>	male recognition, 298
hormones, 106–9	mass communication, 62–63, 214–18
<i>see also</i> exocrine glands	mating, multiple, 155
house (nest site) hunting, 482–92	maze following, 119
Hymenoptera (general), xvi	<i>Megalomyrmex</i> (ants), 457
haplodiploid sex determination, 20–22	<i>Megaponera</i> (ants), <i>see Pachycondyla</i>
<i>Hypoponera</i> (ants), 194, 262, 324, 388	<i>Melipona</i> (stingless bees), 129
	<i>Melophorus</i> (ants), repletes, 257
inclusive fitness, 20–23, 29–42	memory, 117–19, 213
information measurement, 251–52	<i>Messor</i> (harvester ants), 212, 232
intercastes, 388–89	mind, 117–19
<i>see also</i> ergatogynes; ergatoid queens;	<i>Monomorium</i> , 127, 212, 214, 216–17, 292
gamergates	motor displays, 235–47
<i>Iridomyrmex</i> (ants), 266, 280, 288, 321	mound-building ants, 2
Isoptera, <i>see</i> termites	multilevel selection, 7, 7–13, 24–29
	mutilation, ritual, 366–73
juvenile hormone, caste, 106–9, 372	mutualism, <i>see</i> symbioses, ants
	<i>Myanmyrma</i> (fossil ants), 318
kin recognition, 293–98	<i>Myopias</i> (ants), 326
kin selection, 18–19, 23–24, 28–42, 299, 386	

Binary Search

179

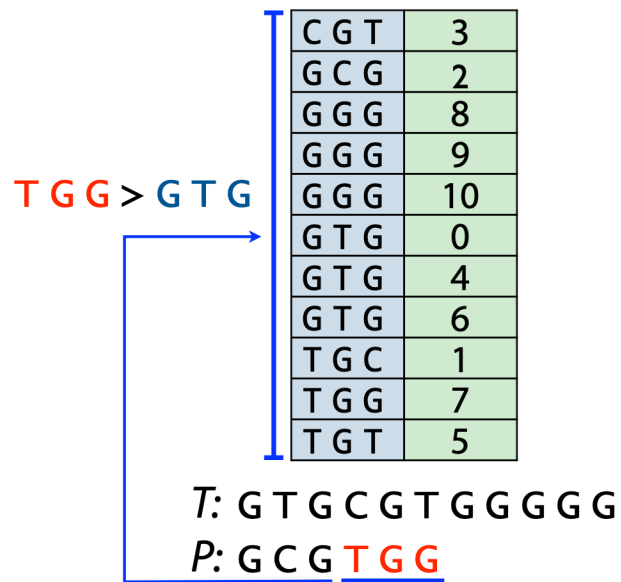
Binary search

C	G	T	3
G	C	G	2
G	G	G	8
G	G	G	9
G	G	G	10
G	T	G	0
G	T	G	4
G	T	G	6
T	G	C	1
T	G	G	7
T	G	T	5

T: G T G C G T G G G G G
 P: G C G T G G

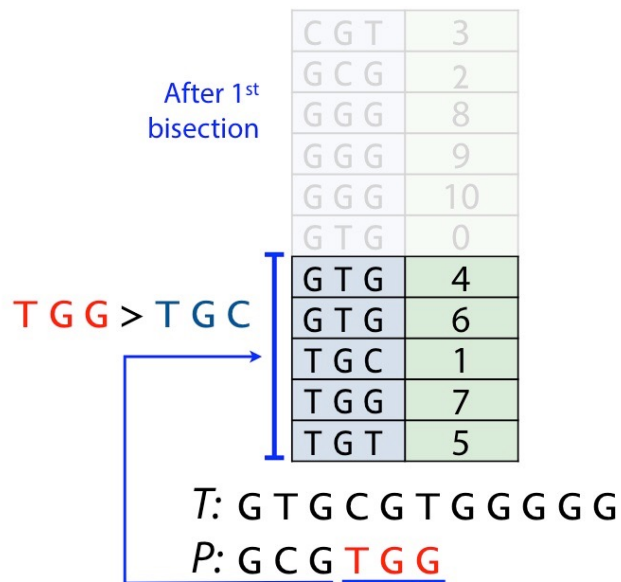
180

Binary search



181

Binary search



182

Binary search

After 2nd bisection

C G T	3
G C G	2
G G G	8
G G G	9
G G G	10
G T G	0
G T G	4
G T G	6
T G C	1
T G G	7
T G T	5

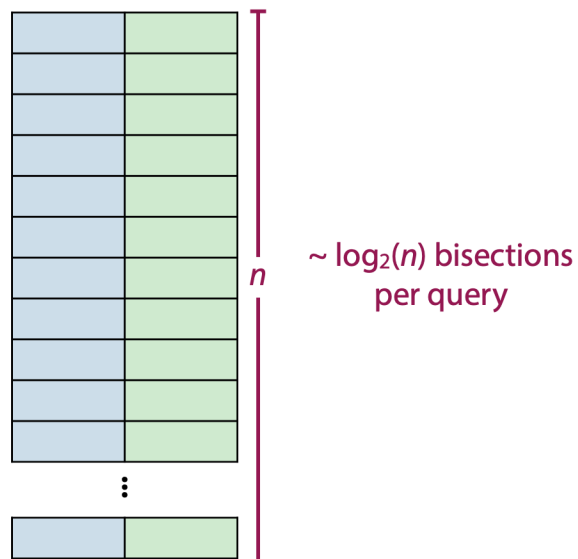
T G G = T G G

T: G T G C G T G G G G G

P: G C G **T G G**

183

Binary search



184

Index of T

CGTGC:	0, 4
GCGTG:	3
GTGCC:	1
GTGCT:	5
TGCCT:	2
TGCTT:	6

T: CGTGCGTGCTT

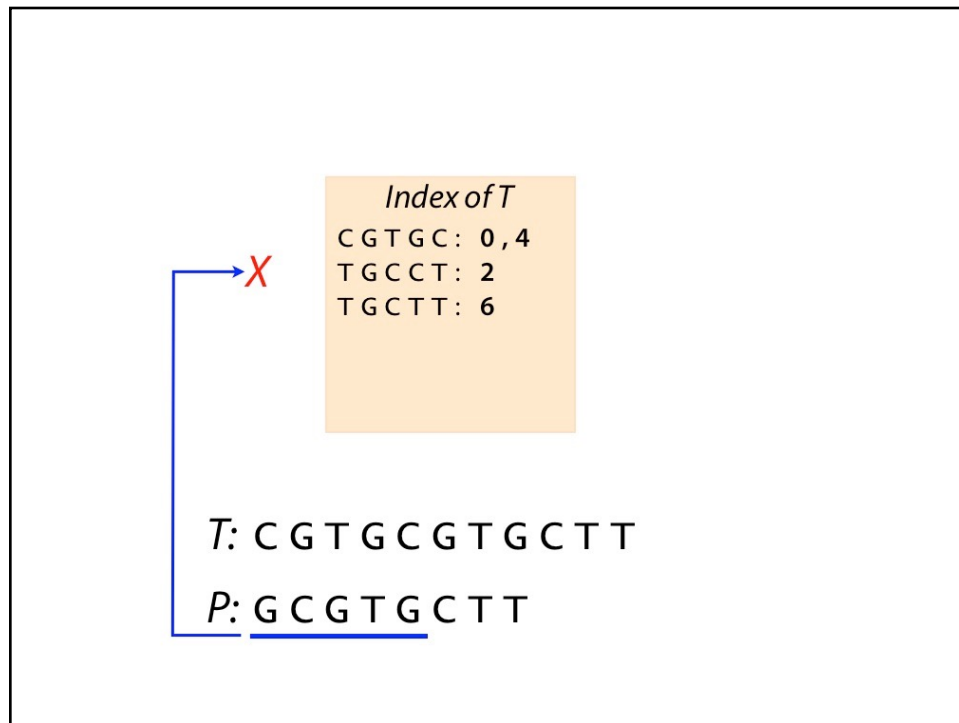
187

Index of T

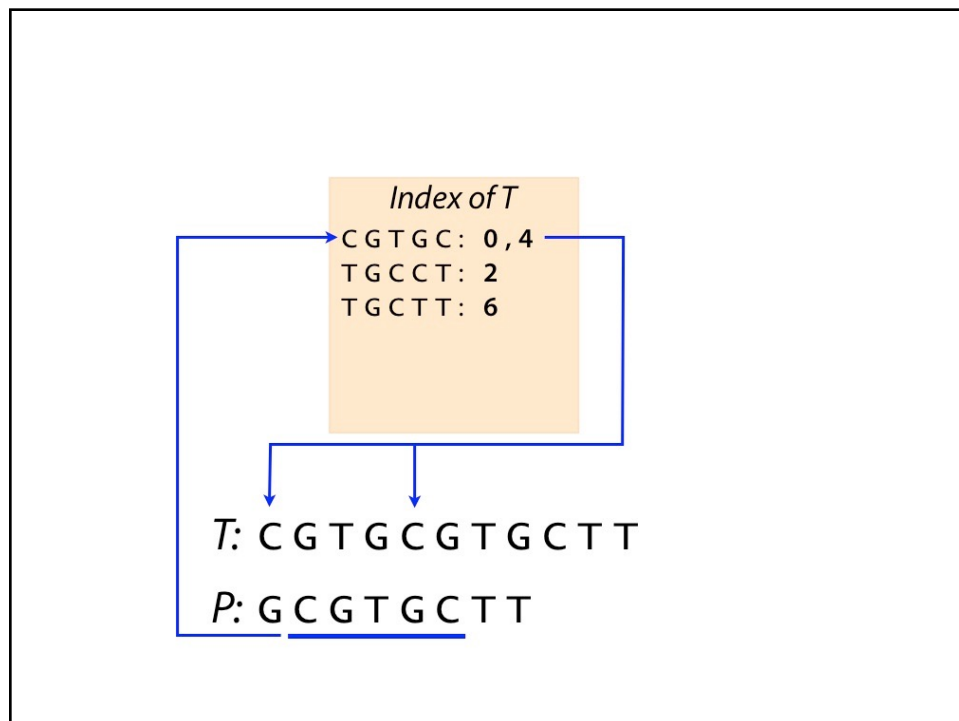
CGTGC:	0, 4
TGCCT:	2
TGCTT:	6

T: CGTGCGTGCTT

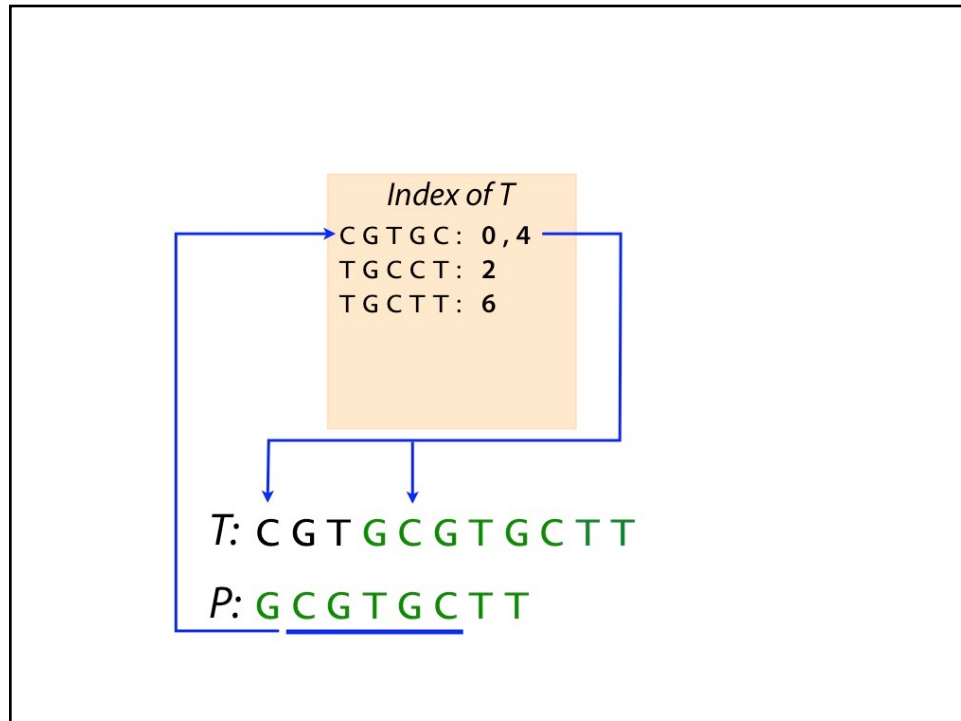
188



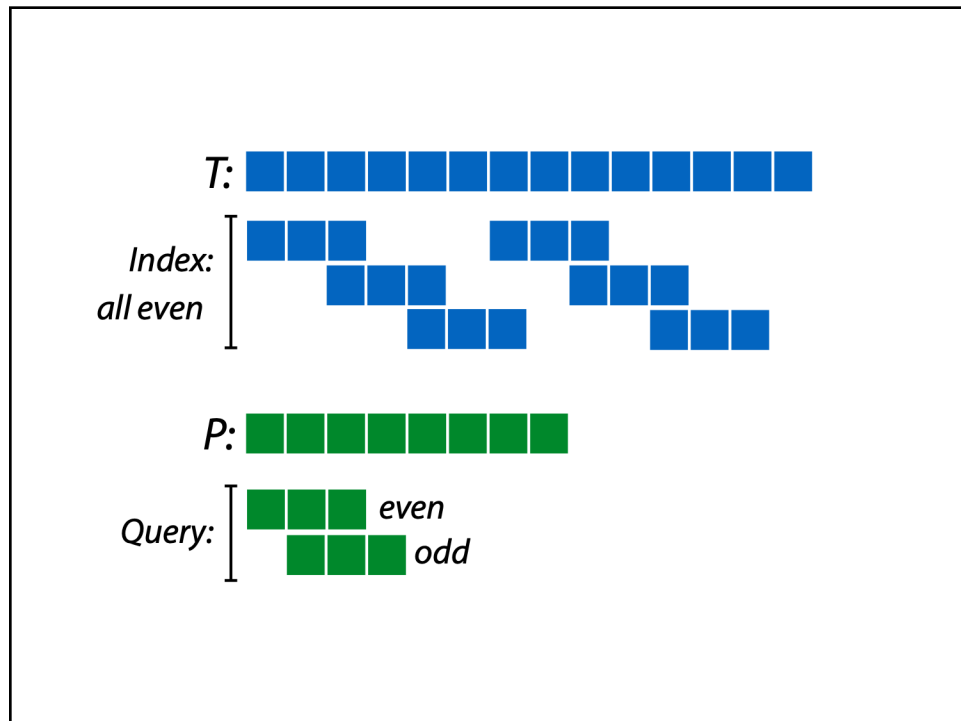
189



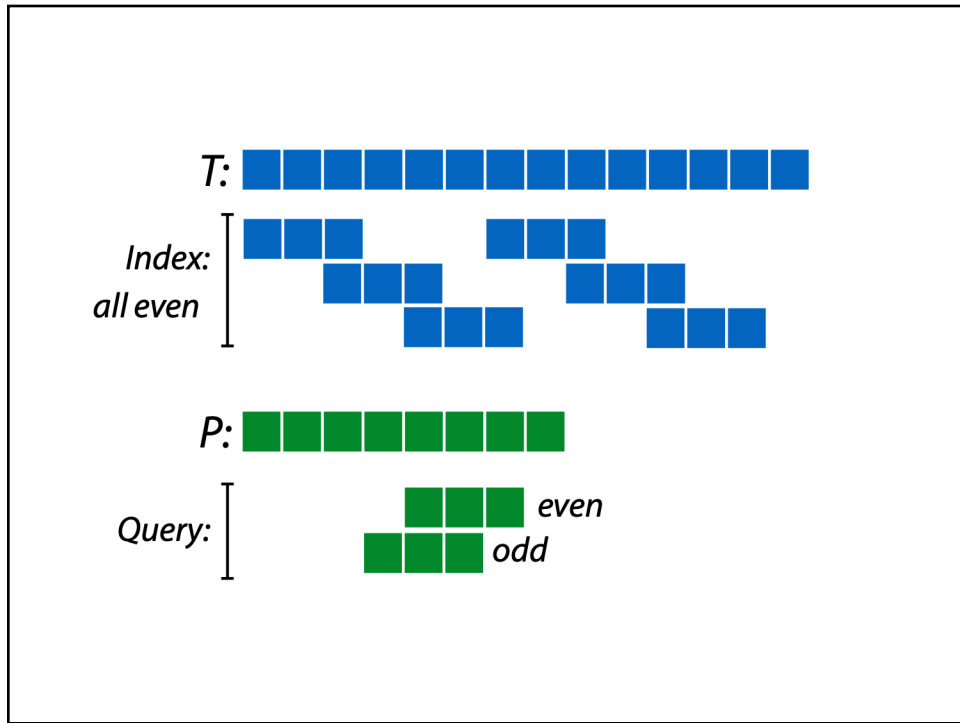
190



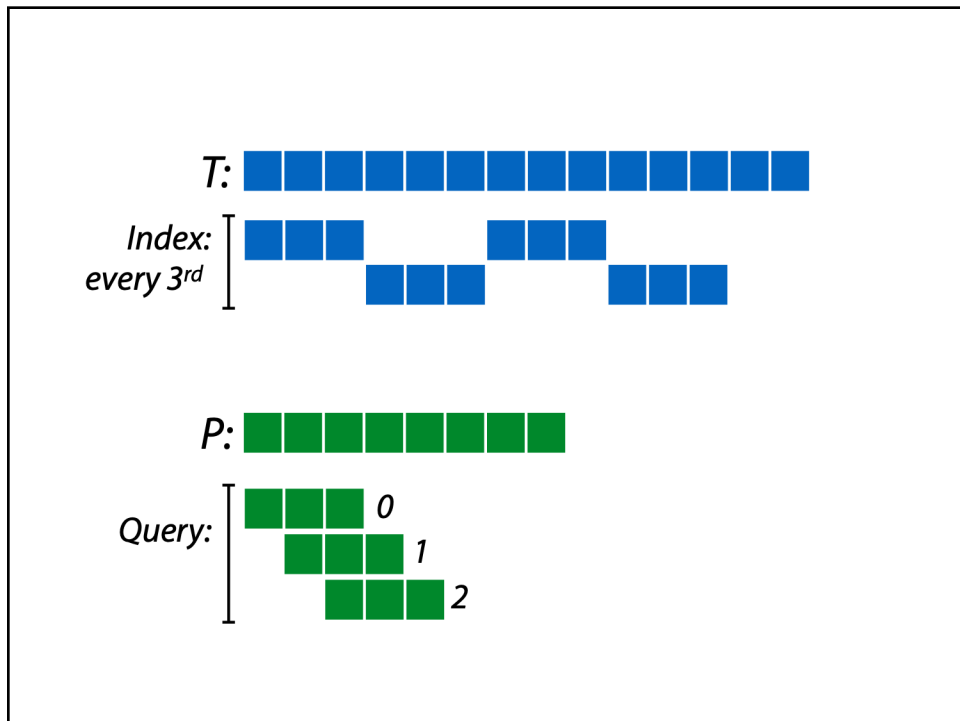
191



192



193



194

Kmer Index Variations -- Subsequence

Index of T
CGGGT: 0

T: CGTGCGTGCTT

195

Index of T
CGGGT: 0
GTCTG: 1

T: CGTGCGTGCTT

196

Index of T

CGGGT:	0
GTCTG:	1
TGGGC:	2

T: C G T G C G T G C T T

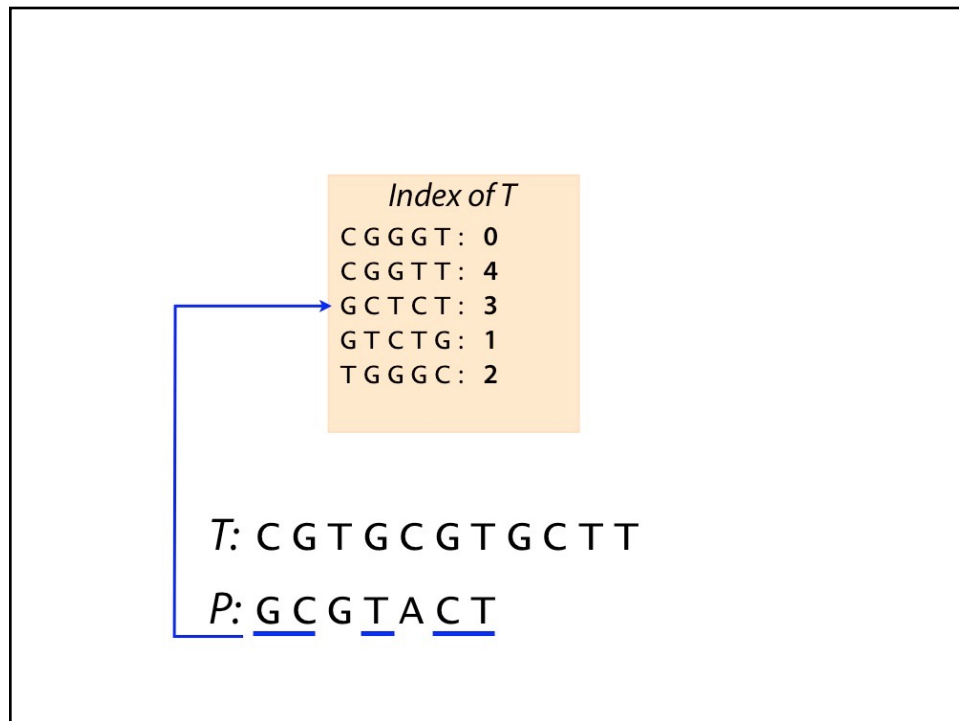
197

Index of T

CGGGT:	0
CGGTT:	4
GCTCT:	3
GTCTG:	1
TGGGC:	2

T: C G T G C G T G C T T

198



199

Suffix index

T : GTTATAGCTGATCGCGGCGTAGCGG
 GTTATAGCTGATCGCGGCGTAGCGG
 TTATAGCTGATCGCGGCGTAGCGG
 TATAGCTGATCGCGGCGTAGCGG
 ATAGCTGATCGCGGCGTAGCGG
 TAGCTGATCGCGGCGTAGCGG
 AGCTGATCGCGGCGTAGCGG
 GCTGATCGCGGCGTAGCGG
 CTGATCGCGGCGTAGCGG
 TGATCGCGGCGTAGCGG
 GATCGCGGCGTAGCGG
 ATCGCGGCGTAGCGG
 TCGCGGCGTAGCGG
 CGCGGCGTAGCGG
 GCGGCGTAGCGG
 CGGCGTAGCGG
 GCGTAGCGG
 CGTAGCGG
 GTAGCGG
 TAGCGG
 AGCGG
 GCGG
 CGG
 GG
 G

200

Suffix index

$T = \text{abaaba}$

abaaba	Alphabetical order 	a
baaba		aaba
aaba		aba
aba		abaaba
ba		ba
a		baaba

201

Suffix index

Querying uses binary search

$P = \text{ab}$ 

a
aaba
aba
abaaba
ba
baaba

202

Suffix index

T : G T T A T A G C T G A T C G C G G C G T A G C G G
 G T T A T A G C T G A T C G C G G C G T A G C G G
 T T A T A G C T G A T C G C G G C G T A G C G G
 T A T A G C T G A T C G C G G C G T A G C G G
 A T A G C T G A T C G C G G C G T A G C G G
 T A G C T G A T C G C G G C G T A G C G G
 A G C T G A T C G C G G C G T A G C G G
 G C T G A T C G C G G C G T A G C G G
 C T G A T C G C G G C G T A G C G G
 T G A T C G C G G C G T A G C G G
 G A T C G C G G C G T A G C G G
 A T C G C G G C G T A G C G G
 T C G C G G C G T A G C G G
 C G C G G C G T A G C G G
 G C G G C G T A G C G G
 C G G C G T A G C G G
 G G C G T A G C G G
 G C G T A G C G G
 C G T A G C G G
 G T A G C G G
 T A G C G G
 A G C G G
 G C G G
 C G G
 G G
 G

$n(n+1)/2 \approx (n^2)/2$
 chars

203

Suffix index

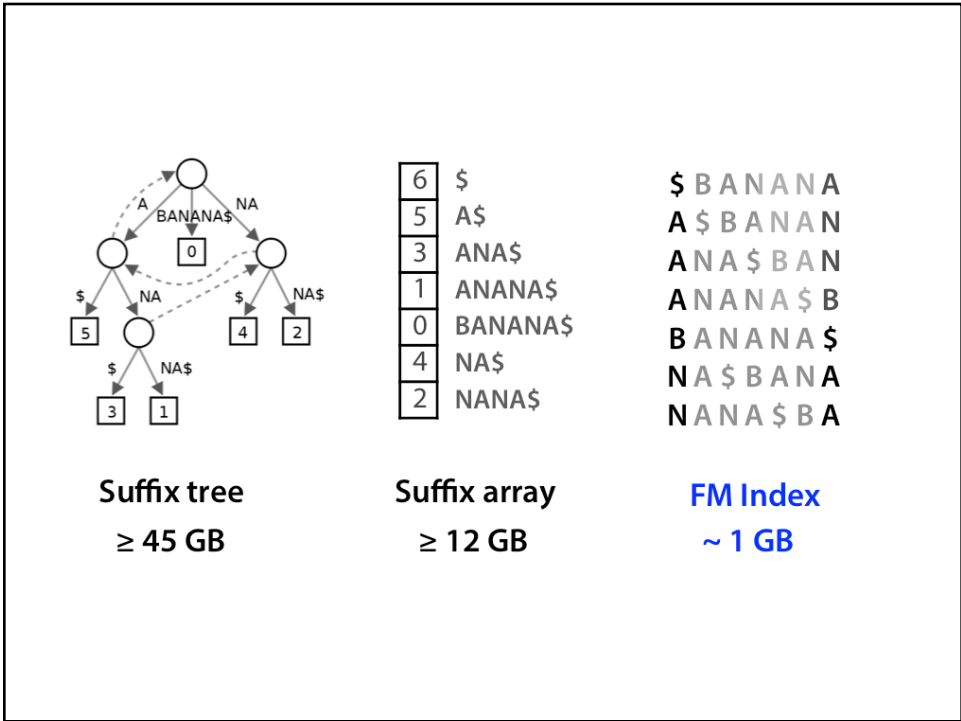
$T = \text{abaaba}$

Suffix array is m
integers long

5	a
2	aaba
3	aba
0	abaaba
4	ba
1	baaba

↑
 SuffixArray(T)

204



205



Software pubmed.ncbi.nlm.nih.gov

Ultrafast and memory-efficient alignment of short DNA sequences to the human genome

Ben Langmead, Cole Trapnell, Mihai Pop and Steven L Salzberg

Address: Center for Bioinformatics and Computational Biology, Institute for Advanced Computer Studies, University of Maryland, College Park, MD 20742, USA.

Correspondence: Ben Langmead. Email: langmead@cbl.umd.edu

Published: 4 March 2009
 Genome Biology 2009, 10:R25 (doi:10.1186/gb-2009-10-3-r25)
 The electronic version of this article is the complete one and can be found online at <http://genomebiology.com/2009/10/3/R25>

Received: 21 October 2008
 Revised: 19 December 2008
 Accepted: 4 March 2009

© 2009 Langmead et al.; licensee BioMed Central Ltd.
 This is an open access article distributed under the terms of the Creative Commons Attribution License (<http://creativecommons.org/licenses/by/2.0>), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.
 Copyright © 2009 Ben Langmead et al. This is an open access article distributed under the terms of the Creative Commons Attribution License (<http://creativecommons.org/licenses/by/2.0>).

Open Access

Fast gapped-read alignment with Bowtie 2

Ben Langmead^{1,2} & Steven L Salzberg¹⁻³

As the rate of sequencing increases, greater throughput is demanded from read aligners. The full-text minute index is often used to make alignment very fast and memory-efficient, but the approach is ill-suited to finding longer, gapped alignments. Bowtie 2 combines the strengths of the full-text minute index with the flexibility and speed of hardware-accelerated dynamic programming algorithms to achieve a combination of high speed, sensitivity and accuracy.

Abstract

Bowtie is an ultrafast, memory-efficient alignment program for aligning short DNA sequence reads to large genomes. For the human genome, Burrows-Wheeler indexing allows Bowtie to align more than 25 million reads per CPU hour with a memory footprint of approximately 1.3 gigabytes. Bowtie extends previous Burrows-Wheeler techniques with a novel quality-aware backtracking algorithm that permits mismatches. Multiple processor cores can be used simultaneously to achieve even greater alignment speeds. Bowtie is open source <http://bowtie.cbcb.umd.edu>.

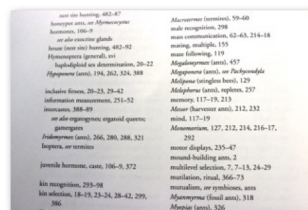
206

Why is it useful?

- Check the accuracy and completeness of *de novo* assemblies
- Map RNASeq data to genomes
- Map metagenomic reads to reference genomes
- Identify insertions

207

Data structures

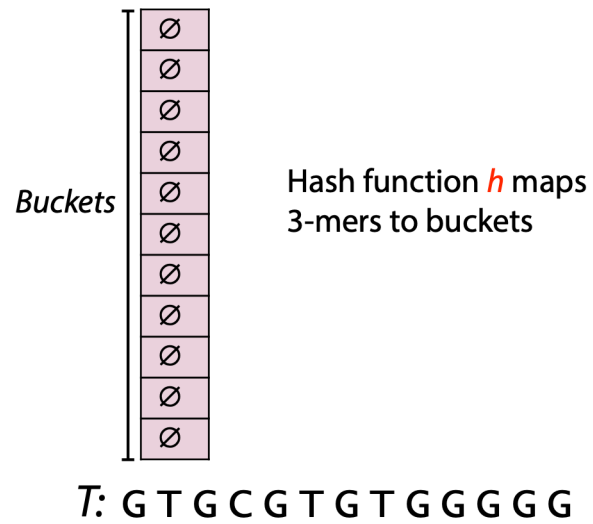


Multimap

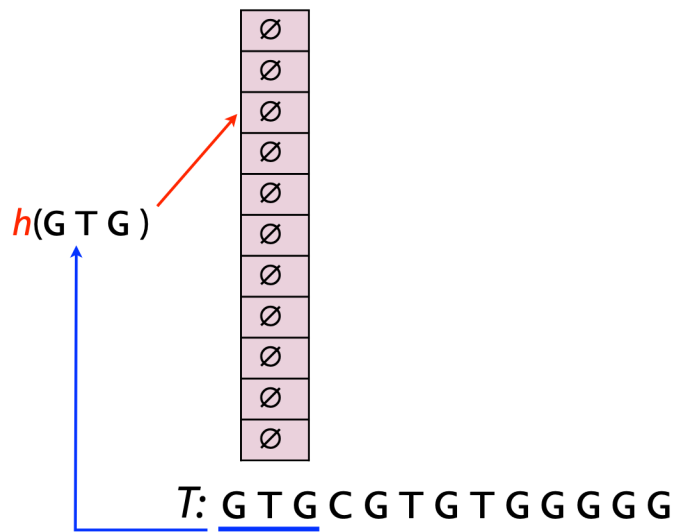
T: C G T G C G T G C T T

208

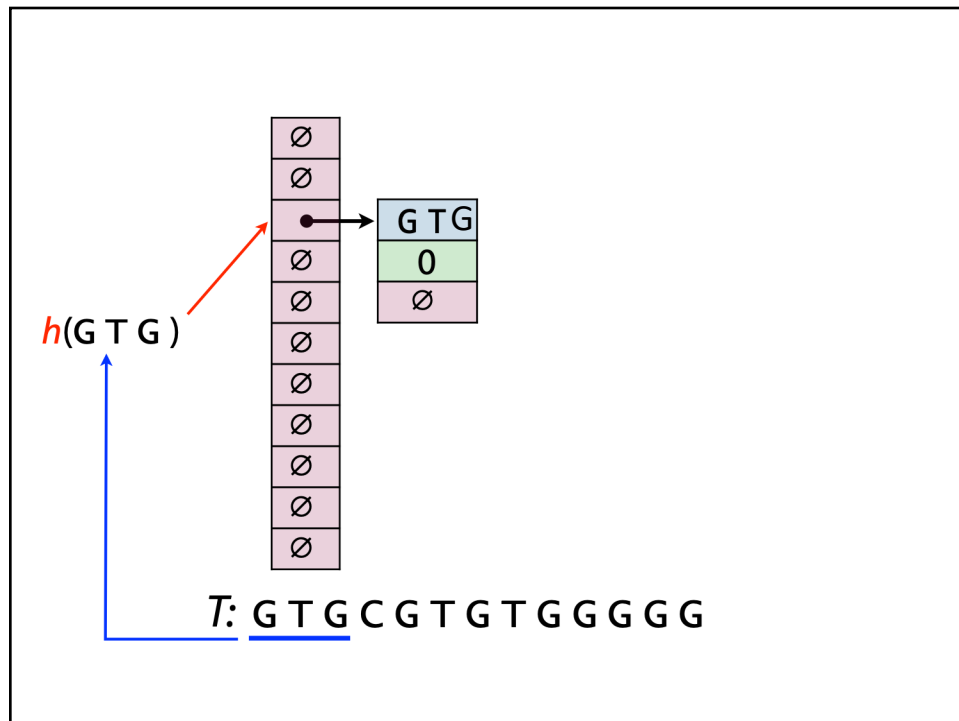
Hash table



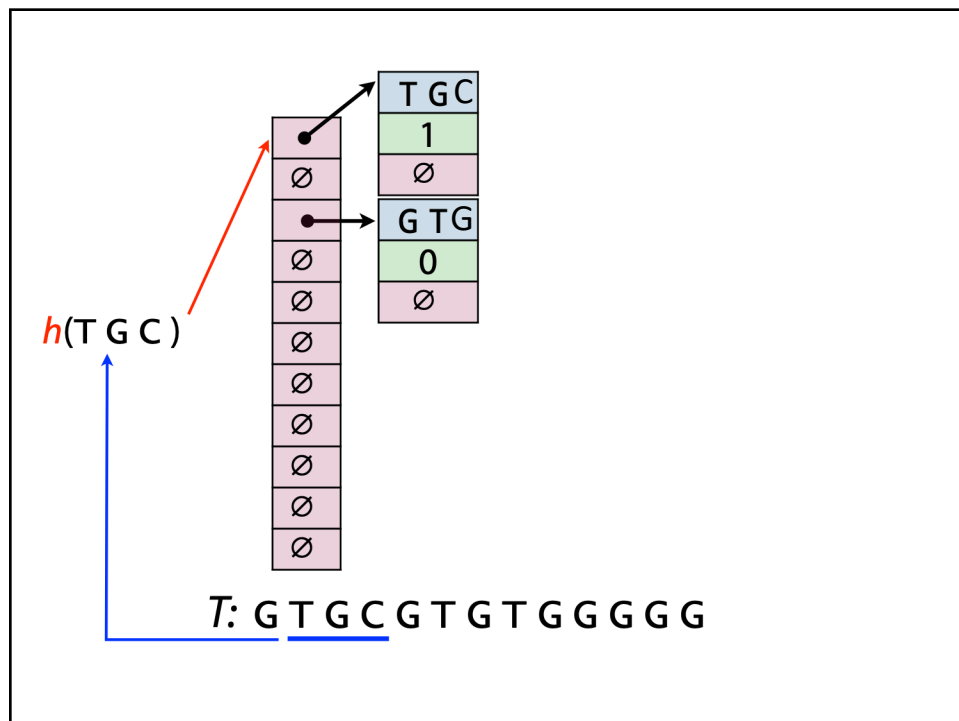
209



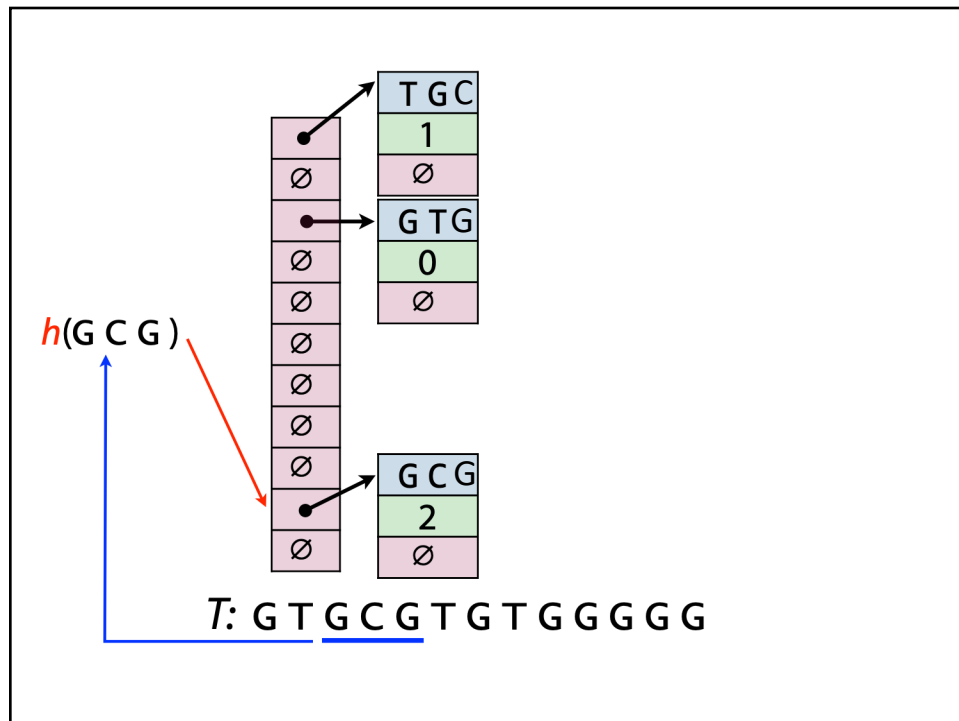
210



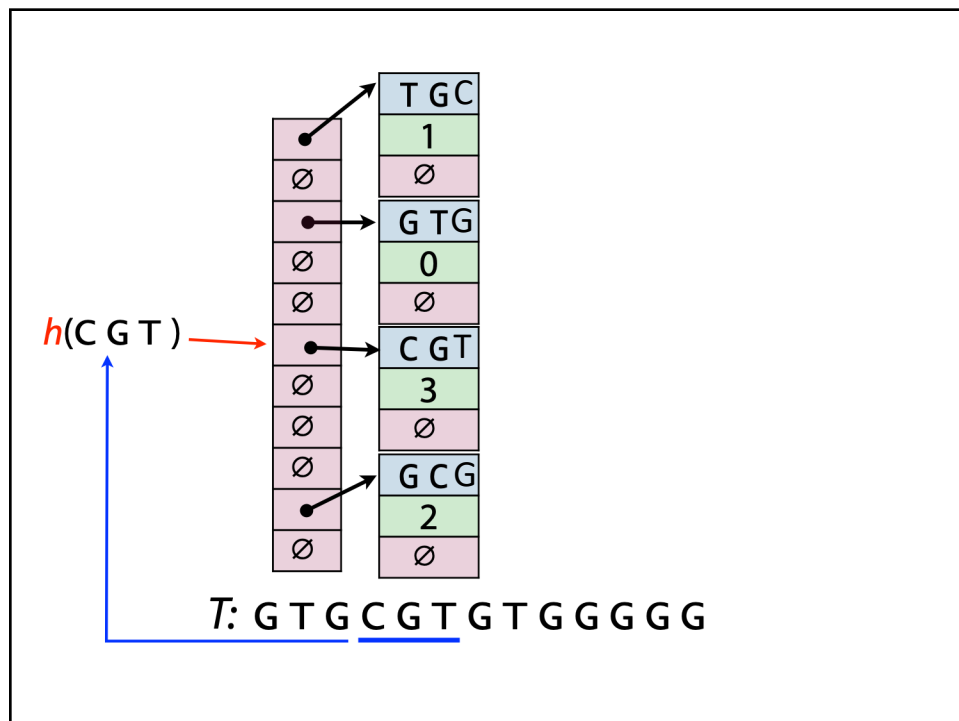
211



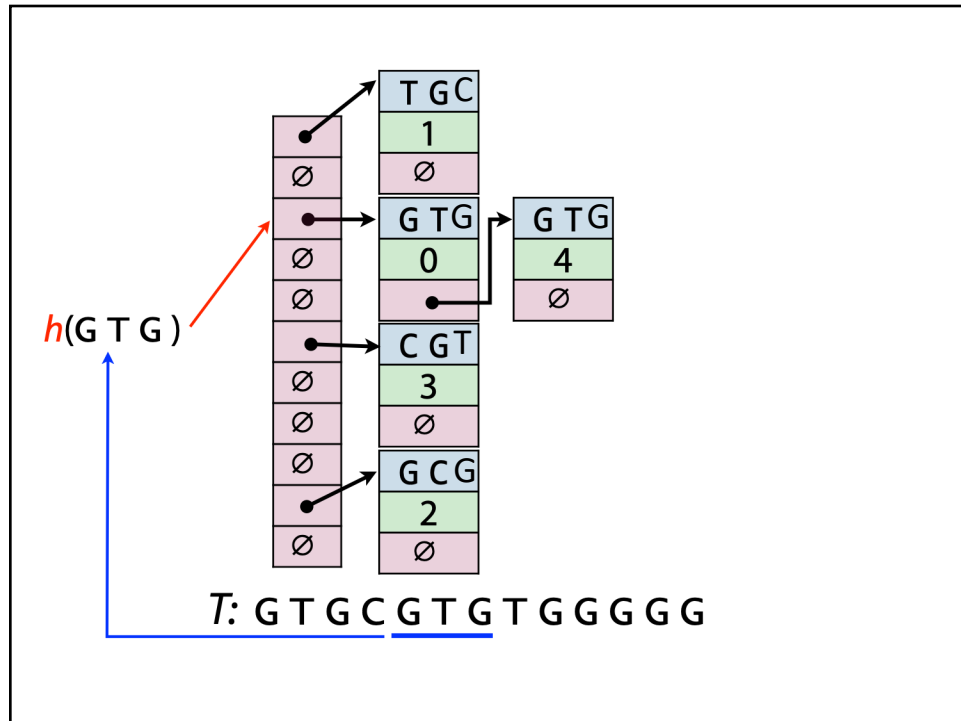
212



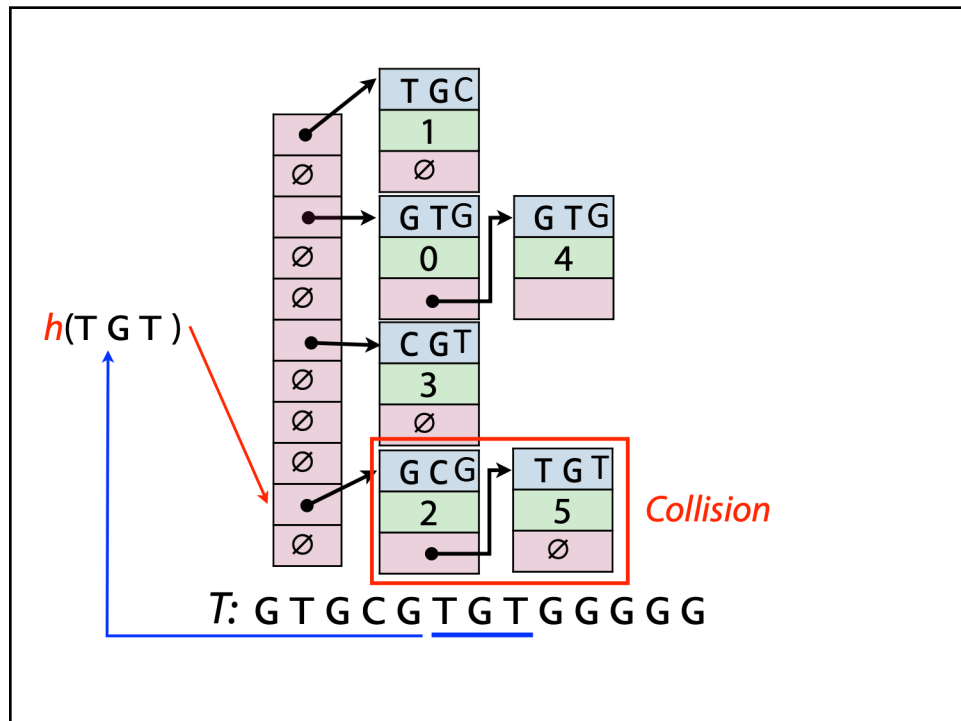
213



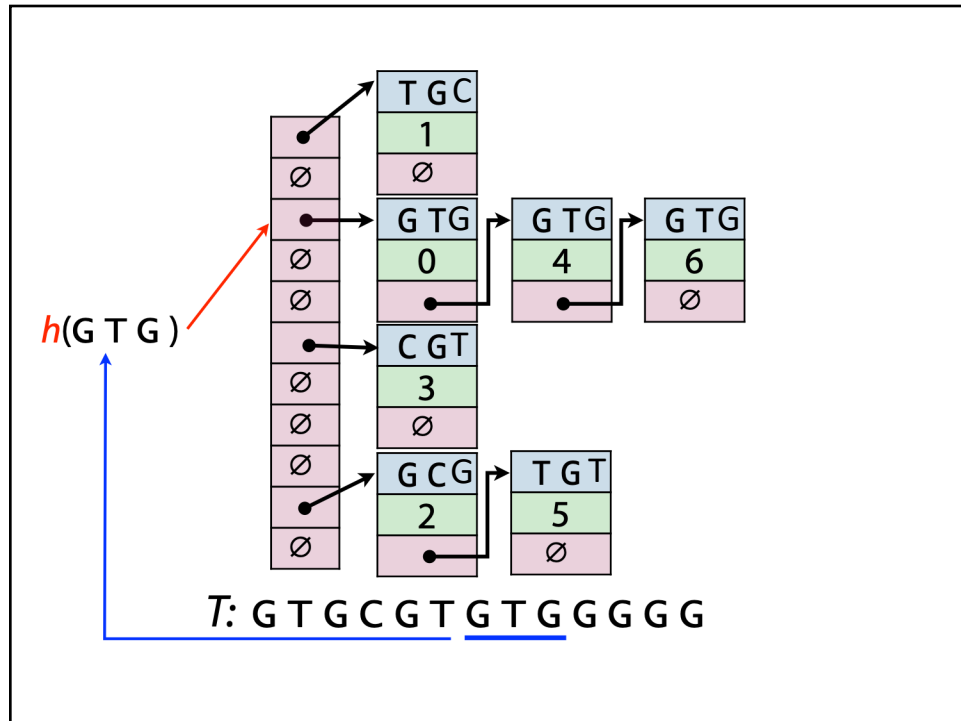
214



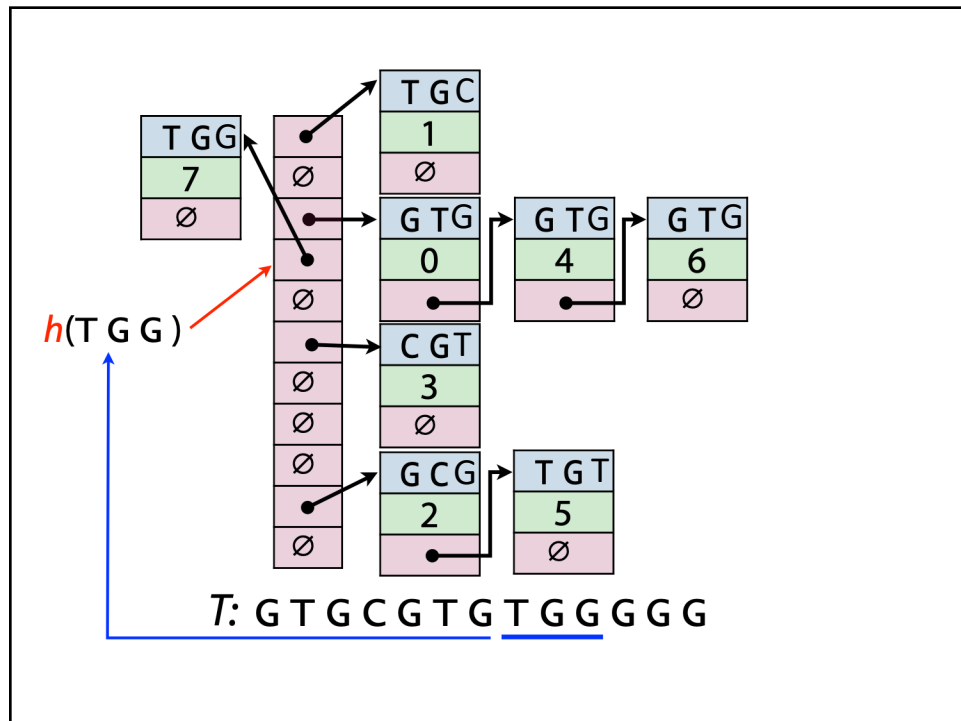
215



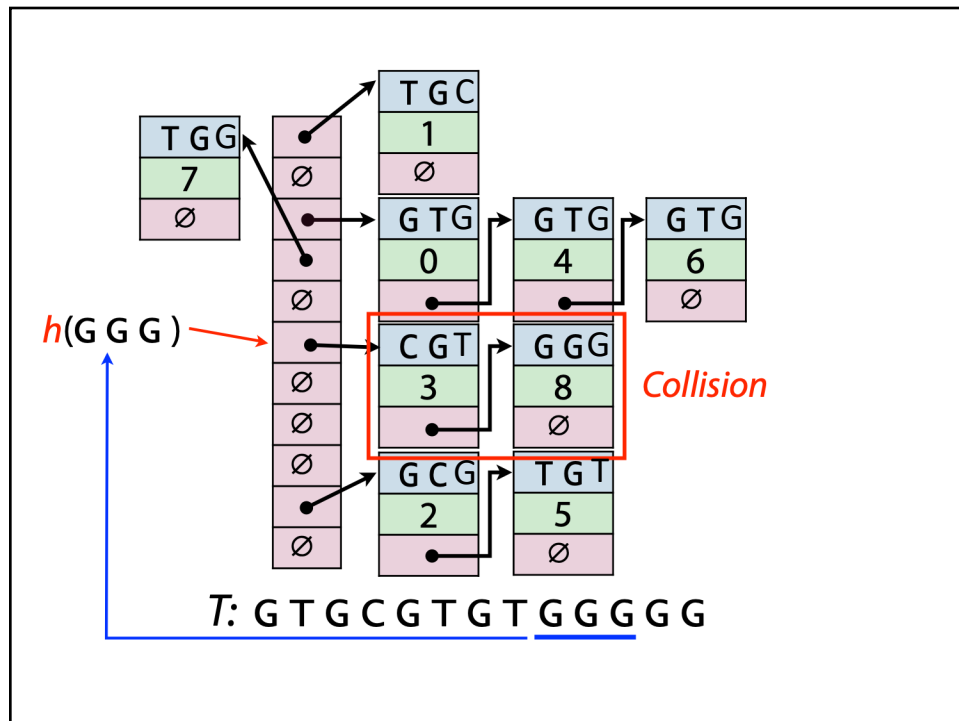
216



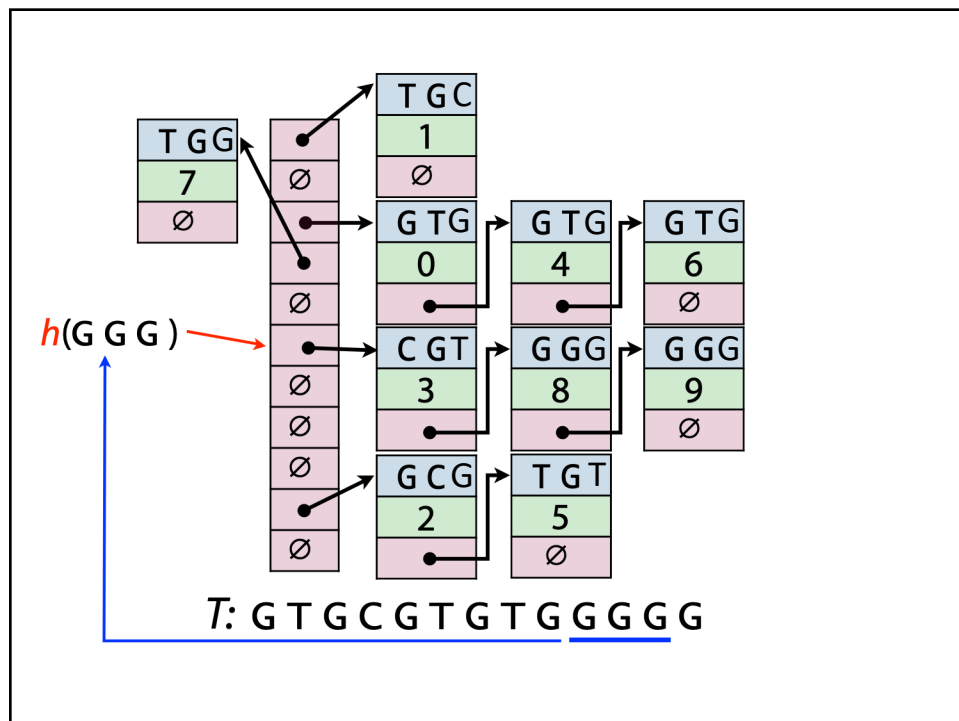
217



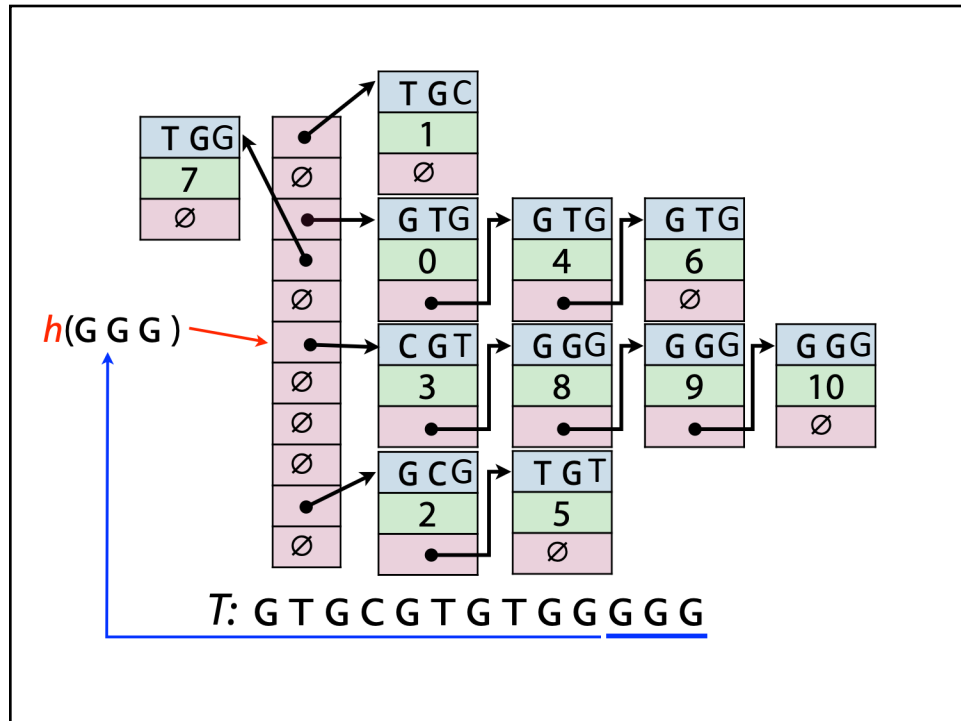
218



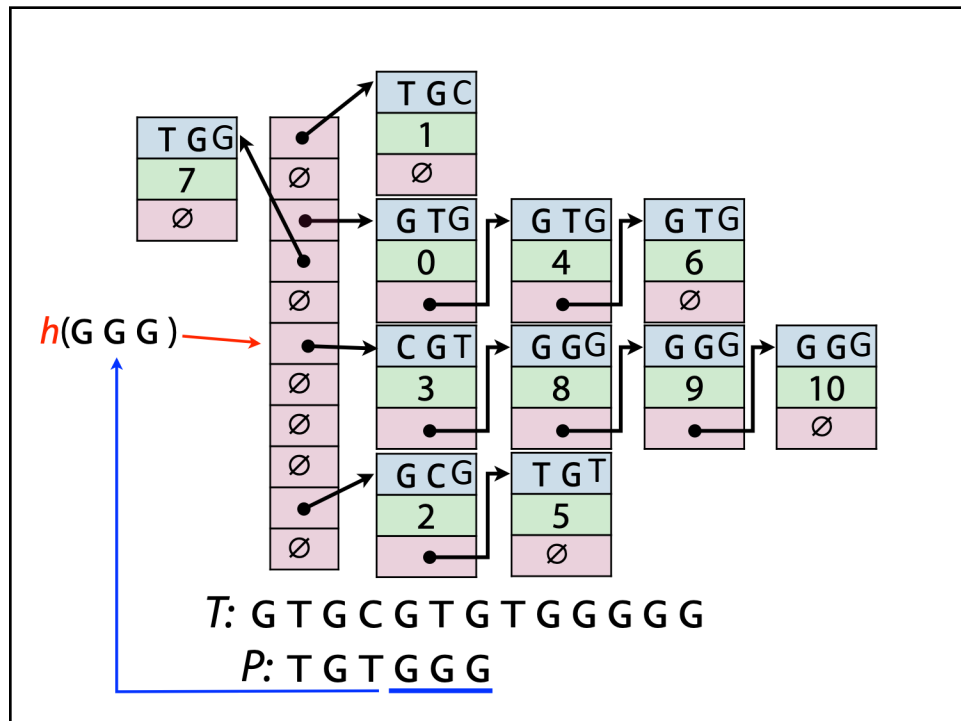
219



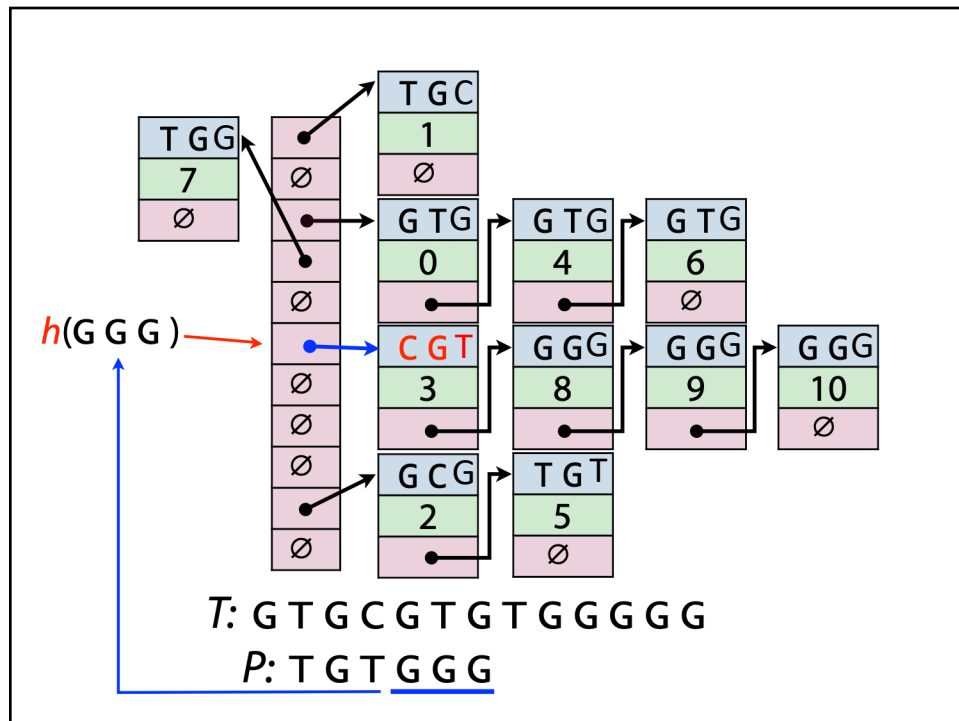
220



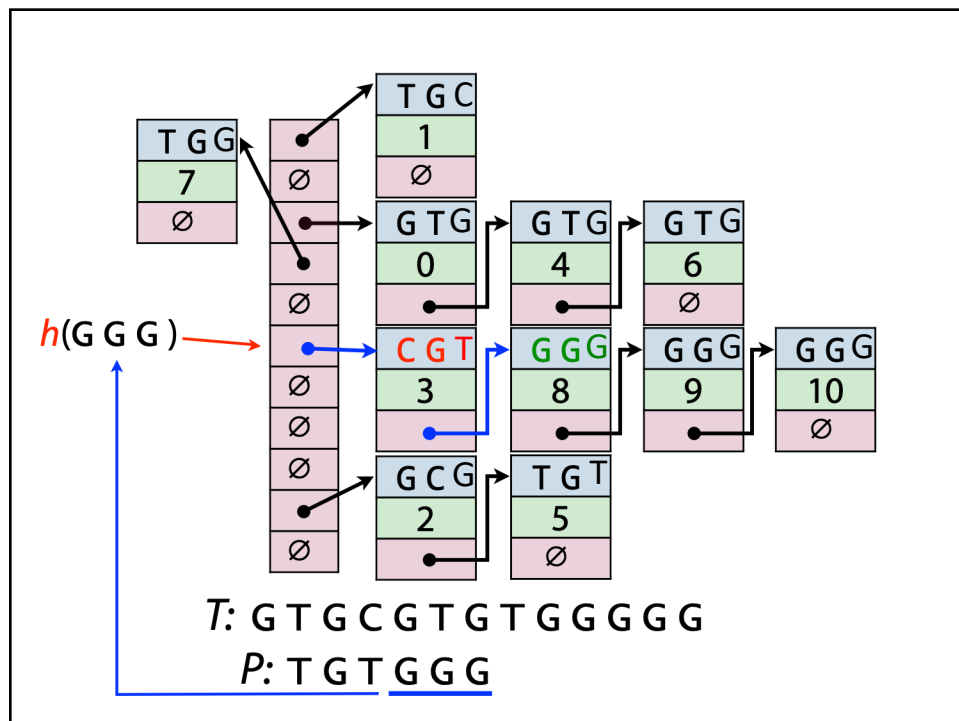
221



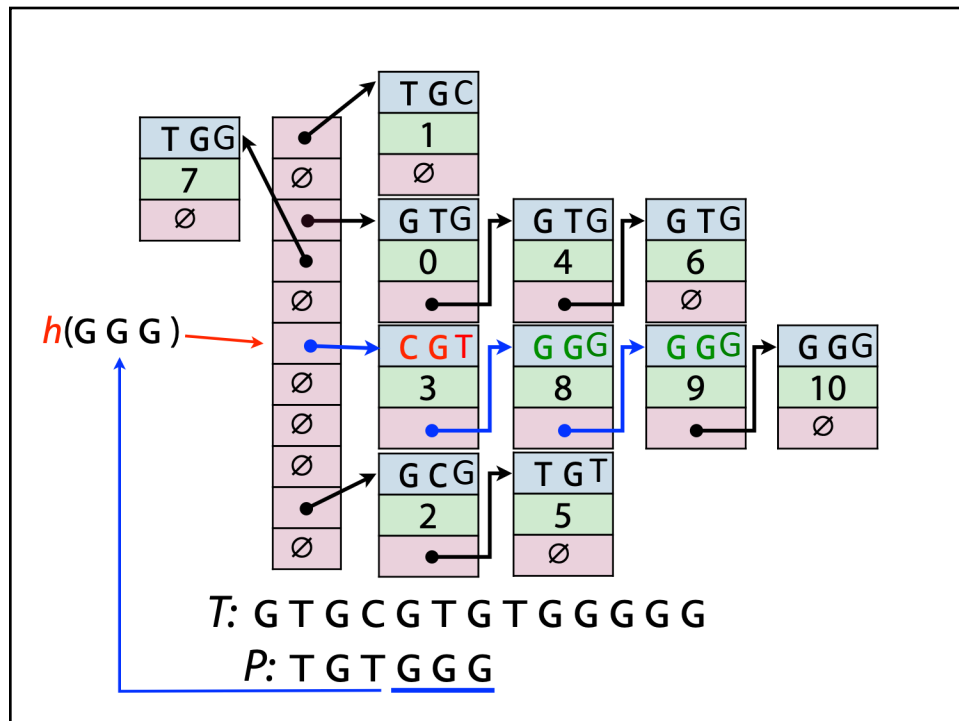
222



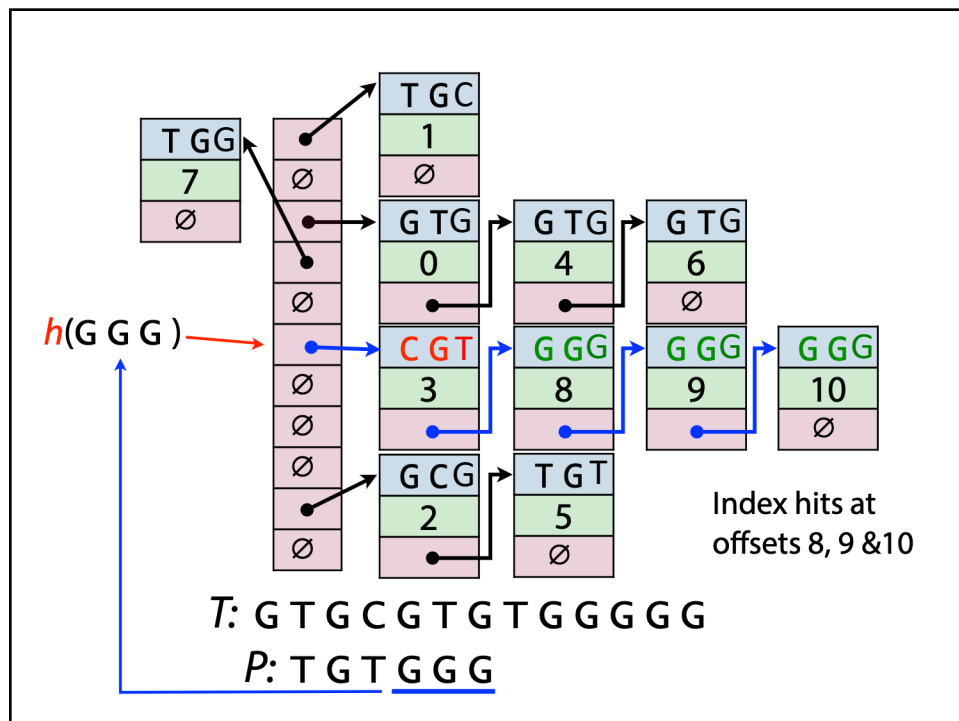
223



224



225



226